

Cerberus - How Radix achieves infinite linear scalability while preserving atomic composability

Transactions

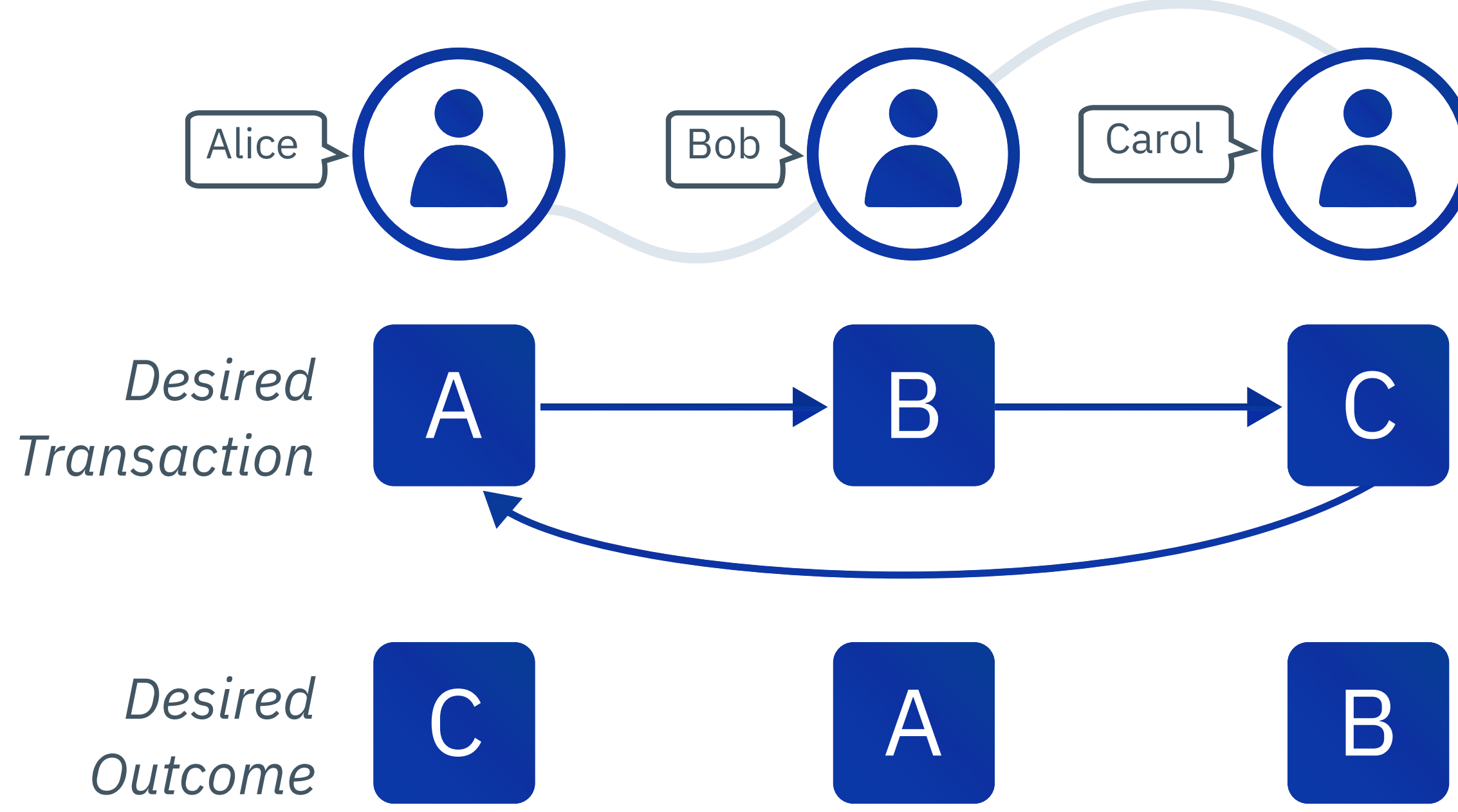
9

So let's walk through how someone would go about exchanging Token A for Token C, using Token B as an intermediary step, all in a single transaction. We will use this transaction for the rest of this infographic series.

In this example Alice owns Token A, Bob Token B, and Carol Token C.

Alice wants Token C, Bob wants Token A, and Carol wants token B.

It's not possible for this transaction to work with just a two way swap. A three way swap is needed.



Client Software "Wallet"

The transaction shown here is exactly the same as the transaction shown below. Here it is presented as text, there it is presented diagrammatically.

Every node in every relevant validator set stores a copy of the whole transaction, including all the substates.

So the first thing that's needed is for Alice, Bob, and Carol to agree on the transaction and that they will all be willing to sign it. This can be done offline.

Once agreed, they use a client, such as a "wallet", to create the transaction. The client software will generate the substates that need to be written to the ledger, and the shards those substates belong in. We covered how substates get allocated shards in the previous episode.

So in this example, three shut down substates are created to record that the tokens, as per the old ownership, can no longer be spent; and three bring up substates are created to record the new owner for each token.

Alice, Bob and Carol then cryptographically sign the transaction proving that they have the authority to send the tokens and that they approve the transaction.

Transaction (Tx)

Shard 1: Shut down Alice's Token A substate

Shard 2: Bring up Bob's Token A substate [Index: 0]

Shard 3: Shut down Bob's Token B substate

Shard 4: Bring up Carol's Token B substate [Index: 1]

Shard 5: Shut down Carol's Token C substate

Shard 6: Bring up Alice's Token C substate [Index: 2]

Signed: Alice, Bob, Carol

Tx ID: ...979

The client thus performs all the computation necessary to define exactly what needs to be written to the ledger - everything is contained in the transaction.

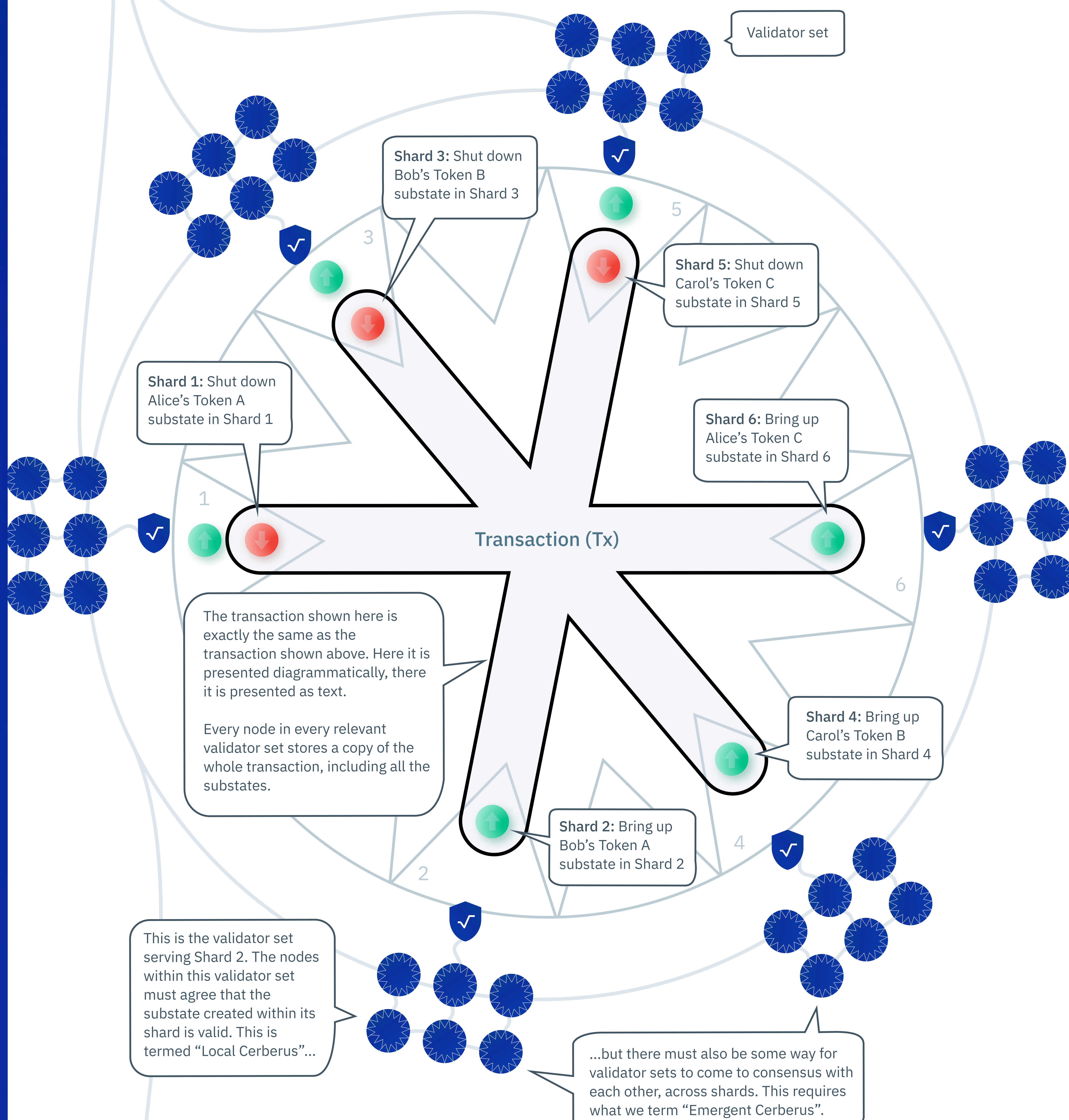
The client then submits the transaction to any node on the network.

Once received by a node, the node can quickly look up the validator sets that serve the shards defined in the transaction, as this information is stored on-ledger. The node can then forward the transaction to the nodes within those validator sets.

The job of a validator set is to verify transactions for the shard it serves, and store a permanent record of the transaction and the substates within it.

So what does a transaction look like?

This allows nodes to easily coordinate, just for this one transaction, no matter where substates reside across the global shardspace.



The above diagram shows six substates being created by six validator sets across six shards, all in one transaction. It contains:

- three shut down substates created to record that the input substates can never be used again in a transaction
- three bring up substates created to record the new owners of the tokens.

But all we've shown here so far is the result of the transaction. We've not described how the nodes within those validator sets communicate with one another and agree that the transaction is valid, in consensus.

To do that, Cerberus' method of reaching consensus is actually composed of two tightly integrated parts that are both

required for the transaction to work.

Those two parts are:

(1) Between the nodes within a validator set for a single shard - this is "Local Cerberus".

(2) Between the validator sets across shards - this is "Emergent Cerberus".

It's important to note that as all transactions are cross-shard, it is actually emergent Cerberus that nodes use to come to consensus. Local Cerberus is just a theoretical stepping stone to help us understand emergent Cerberus.

We start by providing an overview of consensus, and then move on to both Local Cerberus and Emergent Cerberus.

The version of Cerberus described in this infographic series is scheduled to launch as part of the fully sharded Radix Xi'an release. Please visit www.radixdlt.com for details on the Radix roadmap.